

ABSTRACT

Traditional financial systems rely on intermediary institutions such as banks to operate lending services, resulting in high operational costs, lack of transparency, and exclusion of a population segment from credit access. Decentralized Finance (DeFi) has emerged to address these limitations by replacing intermediaries with smart contracts on blockchain. Prominent decentralized lending protocols such as Compound and Aave have demonstrated the viability of this model with tens of billions of USD in total value locked. However, the architecture of these protocols is highly complex, with technical documentation scattered and primarily targeting professional developers, creating significant barriers for academic research and learning.

This thesis adopts the approach of building a complete decentralized lending application on the Ethereum platform, drawing inspiration from the architectures of Compound v2 and Aave v2 while focusing on designing, analyzing, and implementing each component from scratch to elucidate the technical essence of each element rather than reusing existing codebases.

The system is designed with a three-layer architecture. The smart contract layer comprises seven Solidity contracts implementing a Lazy Accrual mechanism combined with a Global Interest Index, a five-layer defense-in-depth security architecture, and the UUPS Proxy pattern. The backend layer employs a multi-process event-driven architecture with five independent workers, uses RabbitMQ ensuring at-least-once event delivery. The frontend layer is built with Next.js, authenticates users via the EIP-4361 standard (Sign-In with Ethereum), and delivers real-time liquidation data updates through WebSocket.

The primary contribution of this thesis is the successful development of a decentralized lending system deployed on the Sepolia testnet and clearly demonstrating the design process and critical technical decisions across each architectural layer. The system was rigorously validated with a total of 198 automated test cases (127 smart contract tests and 71 backend tests, 100% pass rate), achieving over 93% statement code coverage on the smart contract layer.